

Plataforma de avaliação Perguntati- ons



Licenciatura em Eng. Informática
Estágio-Projecto 2023-2024



Tiago Miguel Charrua Pinto

Orientador na empresa: Miguel Barão

Orientador no departamento: Miguel Barão

*Trabalho desenvolvido na empresa Departamento de Informática / Universidade de Évora
no âmbito da disciplina de Estágio-Projeto da Licenciatura em Eng. Informática.*

Évora, 15 de junho de 2025

Conteúdo

1	Introdução	1
1.1	Enquadramento	1
1.2	Objetivos	1
1.3	Contribuições	1
1.4	Estrutura do documento	2
2	Ambiente de desenvolvimento do projeto	3
3	Estado da arte	4
4	Ambiente de desenvolvimento	5
4.1	Ambiente técnico	5
4.1.1	Hardware	5
4.1.2	Ferramentas de desenvolvimento	5
4.1.3	Ambiente aplicacional	6
4.2	Metodologia de trabalho	6
5	Trabalho desenvolvido	7
5.1	Introdução ao Projeto	7
5.2	Desenvolvimento do projeto	8
5.2.1	Implementação dos tipos de questão	8
5.2.2	Implementação do verificador de resposta	12
5.2.3	Substituição do polling por WebSockets na página de administração	13
6	Avaliação crítica	14

1 Introdução

Este relatório foi elaborado no âmbito da Unidade Curricular de Estágio-Projeto da Licenciatura em Engenharia Informática da Universidade de Évora. O estágio decorreu no Departamento de Informática durante um período de três meses e teve como objetivo a melhoria e implementação de novas funcionalidades na plataforma de avaliação *Perguntations*.

Desenvolvida pelo Professor Miguel Barão, docente da Universidade de Évora e meu orientador durante o estágio, a plataforma *Perguntations* permite a criação de provas de avaliação para qualquer Unidade Curricular da universidade. No entanto, apresentava algumas limitações, desde a escassa variedade de tipos de perguntas até questões relacionadas com a página de administração, que necessitavam de ser resolvidas.

Para o desenvolvimento deste projeto, utilizei o Python 3.11 para o *backend* e HTML, JavaScript e CSS [1] para o *frontend*. O código foi escrito no Visual Studio Code, um editor versátil que facilitou a organização e a depuração do trabalho.

Ao longo do estágio, trabalhei maioritariamente de forma autónoma e em regime remoto, realizando reuniões semanais com o professor orientador, no seu gabinete e em horários previamente acordados.

Este relatório está estruturado em várias secções, que descrevem não apenas as atividades desenvolvidas durante o estágio, mas também o enquadramento do projeto, as metodologias utilizadas e as ferramentas empregues.

1.1 Enquadramento

O estágio foi realizado de forma individual e remota, para o Departamento de Informática da Universidade de Évora, no âmbito da unidade curricular Estágio-Projeto da licenciatura de Engenharia Informática. Ao longo do desenvolvimento do projeto, que teve a duração de aproximadamente três meses, contei com o acompanhamento e orientação do Professor Miguel Barão, tanto em formato presencial como remoto, em horários acordados entre ambas as partes.

1.2 Objetivos

Este projeto tem como objetivo melhorar a plataforma *Perguntations*, utilizada para a realização de avaliações na Universidade de Évora. Originalmente desenvolvida pelo Professor Miguel Barão, a plataforma apresentava algumas limitações e funcionalidades incompletas. Deste modo, o propósito deste trabalho consistiu em corrigir essas falhas e expandir as suas capacidades, tornando-a mais eficiente e funcional.

1.3 Contribuições

No início do estágio, em colaboração com o professor orientador, estabelecemos um conjunto de objetivos a concretizar ao longo do projeto. Embora a maioria tenha sido alcançada, alguns não foram totalmente concluídos devido ao tempo limitado. No entanto, as metas mais relevantes foram plenamente atingidas, nomeadamente:

- Implementação de novos formatos de questões: correspondência, preenchimento de lacunas e ordenação;
- Substituição do sistema de polling por WebSockets [2] na página de administração, garantindo maior eficiência;
- Introdução da funcionalidade de verificação automática da correção em questões de código durante o teste.

1.4 Estrutura do documento

O presente documento encontra-se organizado em várias secções, conforme descrito abaixo:

1. **Ambiente de Desenvolvimento do Projeto** [2] - Esta secção descreve a organização onde decorreu o estágio, o processo de integração e a dinâmica de trabalho colaborativo com o professor orientador.
2. **Estado da Arte** [3] - Nesta secção, é realizada uma análise comparativa entre a plataforma *Perguntations* e outras soluções existentes com objetivos semelhantes, destacando-se as vantagens competitivas da primeira.
3. **Ambiente de Desenvolvimento** [4] - Esta parte detalha o ambiente de hardware e software utilizado durante o estágio, incluindo ferramentas e tecnologias empregues.
4. **Trabalho Desenvolvido** [5] - A secção inicia-se com um resumo conciso das atividades realizadas ao longo do estágio, seguido de uma descrição pormenorizada do trabalho. Esta divide-se em quatro subsecções; uma para a introdução e três dedicadas a cada uma das contribuições específicas para o projeto.
5. **Avaliação Crítica** [6] - Por fim, apresenta-se uma reflexão pessoal sobre o decorrer do estágio, as competências adquiridas e um balanço global da experiência.

2 Ambiente de desenvolvimento do projeto

O Departamento de Informática da Universidade de Évora constitui uma unidade científico-pedagógica da Universidade de Évora, dedicada à investigação, ao ensino e à prestação de serviços especializados à comunidade. Atua nas áreas da Informática e em domínios afins ou interdisciplinares, promovendo a inovação e a excelência académica.

Durante o estágio, participei no desenvolvimento remoto da plataforma *Perguntations*, uma ferramenta concebida para a criação e aplicação de provas de avaliação destinadas às unidades curriculares da Universidade de Évora. O projeto foi inicialmente desenvolvido pelo Professor Miguel Barão, do Departamento de Informática, que me orientou e acompanhou ao longo de todo o processo.

Apesar de ter trabalhado maioritariamente de forma autónoma, realizei reuniões semanais presenciais com o professor orientador, no seu gabinete e em horários acordados entre ambas as partes. Estas sessões permitiram discutir o estado atual do projeto, receber *feedback* detalhado sobre o trabalho desenvolvido e definir as prioridades seguintes, como a implementação de novas funcionalidades ou o aperfeiçoamento de aspetos já existentes.

Para garantir uma gestão eficiente do código, utilizei o Git como sistema de controlo de versões. Esta ferramenta foi essencial para manter um registo completo de todas as alterações realizadas, assim como para partilhar o projeto de forma segura através de um repositório privado. Desta forma, o professor orientador pôde acompanhar o meu progresso de maneira clara e organizada, contribuindo com sugestões sempre que necessário.

3 Estado da arte

Nesta secção, apresenta-se uma análise comparativa entre a plataforma *Perguntations* e outras soluções existentes para a realização de provas de avaliação em ambiente digital. O objetivo é contextualizar o *Perguntations* no panorama atual, destacando as suas particularidades, vantagens e eventuais limitações face às alternativas disponíveis.

A plataforma distingue-se pelas seguintes funcionalidades:

- Funcionamento exclusivo em rede local (sem acesso à Internet);
- Compatibilidade universal com qualquer sistema operativo e navegador;
- Suporte a diversos formatos de questões: escolha múltipla, correspondência, ordenação, arrastar e soltar, perguntas de desenvolvimento e perguntas de código;
- Funções de monitorização, como verificação de ecrã completo ou deteção de saída da página;
- Possibilidade de revisão da prova pelo aluno;
- Maior segurança, com reduzida vulnerabilidade a fraudes.

Atualmente, existem outras plataformas amplamente utilizadas em contexto académico, como o Moodle e o Google Forms, que apresentam algumas limitações em comparação com o *Perguntations*:

- Acesso através da Internet, o que aumenta o risco de fraude;
- Menor variedade de tipos de questões disponíveis;
- Ausência de funções de monitorização em tempo real;
- Maior exposição a tentativas de fraude, devido à dependência da Internet e à falta de mecanismos de controlo robustos.

Esta análise permite concluir que o *Perguntations* oferece uma solução mais segura e adaptada a ambientes controlados, embora com a limitação inerente ao requisito de rede local.

4 Ambiente de desenvolvimento

A nível de hardware, trabalhei com o meu computador pessoal para o desenvolvimento do projeto.

A nível de software, utilizei um conjunto de ferramentas modernas e eficientes. Para o *backend* e processamento de dados, trabalhei com o Python 3.11 com bibliotecas como Argon2 [3], Mistune [4], PyYAML [5], Pygments [6], SQLAlchemy [7] e Tornado [8]. No *frontend*, utilizei HTML, CSS e JavaScript, combinados com Bootstrap [9], CodeMirror [10], DataTables [11], jQuery [12], MathJax [13] e SortableJS [14]. Além disso, recorri ao Visual Studio Code (VSCode) e ao Git.

4.1 Ambiente técnico

4.1.1 Hardware

O meu computador pessoal é um portátil Lenovo, com um processador AMD Ryzen 7 5700U, 16 GB de memória RAM, com placa gráfica integrada Radeon Graphics e um armazenamento rápido de 512 GB SSD.

O computador tem dual-boot com Windows 11 e Ubuntu 22.04, sendo que o Ubuntu foi o sistema operacional utilizado no desenvolvimento do projeto.

4.1.2 Ferramentas de desenvolvimento

Para o desenvolvimento deste projeto, utilizei a linguagem Python 3.11 para o *backend* e HTML, JavaScript e CSS para o *frontend*. O código foi escrito no Visual Studio Code, um editor de código versátil que facilitou a organização e a depuração do projeto.

Antes de iniciar o desenvolvimento, foi necessário configurar um ambiente virtual (virtual environment) dentro da pasta do projeto. Isto permitiu-me instalar a versão específica do Python (3.11) e gerir as dependências do projeto de forma isolada.

Para o *backend* e processamento de dados, recorri às seguintes bibliotecas:

- Argon2 (hashing seguro de passwords);
- Mistune (renderização de Markdown);
- PyYAML (manipulação de ficheiros YAML);
- Pygments (syntax highlighting para código);
- SQLAlchemy (ORM (Object-Relational Mapping) para bases de dados).

No *frontend*, trabalhei com HTML, CSS e JavaScript, complementados por:

- Bootstrap (framework para design responsivo);
- CodeMirror (editor de código no navegador);
- DataTables (tabelas interativas e pesquisáveis);
- jQuery (simplificação de manipulação DOM [15] e AJAX [16]);
- MathJax (renderização de fórmulas matemáticas);
- Sortable.js (arrastar e soltar elementos).

Para instalar as bibliotecas do Python, utilizei o pip (gestor de pacotes do Python). Para as dependências do JavaScript, utilizei o npm (Node Package Manager).

Foi utilizada a framework Tornado para desenvolver um servidor web assíncrono de alto desempenho, tirando partido da sua arquitetura não bloqueante para gerir múltiplas ligações simultâneas sem bloqueios de I/O. Esta opção revelou-se ideal para suportar um elevado número de utilizadores concorrentes, como alunos e administradores durante um teste online.

Além disso, utilizei a linguagem YAML [17] para a configuração dos testes e perguntas. O YAML permitiu estruturar de forma clara e legível os dados dos testes, incluindo a definição de perguntas, opções de resposta, cotações e critérios de correção.

Para organizar e acompanhar o desenvolvimento do código, utilizei o Git como sistema de controlo de versões. Esta ferramenta permitiu manter um histórico completo de todas as alterações, além de facilitar a partilha segura do projeto através de um repositório privado. Desta forma, o professor pôde acompanhar o progresso de forma eficiente.

4.1.3 Ambiente aplicacional

Este projeto consiste na melhoria e expansão da plataforma de avaliação *Perguntations*, desenvolvida inicialmente pelo professor Miguel Barão, para uso individual numa Unidade Curricular da universidade. Trata-se de um software *standalone* que opera numa rede local configurada pelo docente, permitindo que os alunos acessem às suas funcionalidades através de um link partilhado. Durante a utilização, os alunos – que ficam sem acesso à Internet enquanto estão conectados a esta rede – são monitorizados pela plataforma, que deteta eventuais saídas da página ou alterações no tamanho do ecrã.

O ambiente atual de operação requer apenas um servidor com capacidade de criar os testes dos alunos, como também receber as submissões dos mesmos. A plataforma é compatível com os principais navegadores web e sistemas operativos, garantindo acessibilidade sem requisitos complexos por parte dos utilizadores.

A plataforma funciona de forma autónoma, não dependendo de sistemas externos da universidade.

4.2 Metodologia de trabalho

No desenvolvimento do projeto, foi adotada a metodologia de Desenvolvimento Ágil de Software [18], que se baseia num processo iterativo e incremental, com ciclos curtos de trabalho (*sprints*), visando reduzir riscos e privilegiar a comunicação em tempo real. Em cada iteração, eram definidas novas funcionalidades (*features*) a implementar na plataforma. No final de cada ciclo, avaliavam-se os resultados e, na semana seguinte, prosseguia-se com a definição de novas tarefas, correção de erros ou melhorias no que havia sido desenvolvido anteriormente.

Trabalhei individualmente, mas realizei reuniões semanais com o professor coordenador para alinhar o ponto de situação do projeto. Essas reuniões serviam para obter *feedback* do professor, bem como para definir novas funcionalidades a implementar ou melhorias no que já estava desenvolvido.

Para manter tudo atualizado, fazia sempre um *push* no GitHub assim que uma tarefa ficava concluída. Algumas tarefas foram finalizadas numa semana, enquanto outras, devido à sua complexidade, demoraram mais tempo.

5 Trabalho desenvolvido

No início do estágio, reuni-me com o professor Miguel Barão que foi o meu orientador durante todo o período, para uma apresentação inicial sobre o projeto e a plataforma *Perguntations*. Durante o encontro, ele destacou algumas funcionalidades e problemas existentes na plataforma e sugeriu que, na semana seguinte, eu falasse com colegas para recolher opiniões sobre possíveis melhorias. O objetivo era reunir o máximo de informações para orientar as implementações futuras. Após a recolha de informações, o professor disponibilizou-me o código da plataforma para que eu pudesse analisá-lo e começar a trabalhar. Para tal, foi necessário instalar algumas ferramentas que permitissem executar a plataforma localmente.

Depois de estudar a estrutura do código, iniciei as alterações pelo *frontend*, implementando novos tipos de questões. Comecei pelas questões de correspondência, que representaram um desafio inicial devido à minha pouca familiaridade com as ferramentas utilizadas. Apesar dessa dificuldade, consegui concluir a funcionalidade com sucesso. Seguiram-se, por esta ordem, as implementações de:

- Questões de ordenação;
- Questões de preenchimento de lacunas (dois tipos distintos);
- Questões de correspondência com linhas.

Embora cada tipo apresentasse os seus próprios obstáculos, o processo tornou-se progressivamente mais fluído, à medida que ganhava familiaridade com o código. No final, em concordância com o professor, considerou-se que as implementações estavam concluídas e funcionais.

Posteriormente, avancei para o *backend*, que representou a parte mais complexa do projeto. O primeiro desafio consistiu na implementação de um botão que, ao ser acionado, avalia uma questão de código e fornece *feedback* ao utilizador - indicando se a implementação está correta ou se contém erros de sintaxe ou nos testes.

Nesta fase, deparei-me com algumas dificuldades técnicas que impediram a implementação exata da funcionalidade inicialmente planeada: o desconto automático do valor da questão quando o botão fosse utilizado e o *feedback* fosse negativo. Como solução alternativa, implementei um sistema de limite de utilizações ajustável para cada questão, que cumpria o mesmo propósito de forma eficaz.

A última etapa do projeto focou-se na página de administração, onde substituí o sistema de polling por WebSockets. Esta alteração permitiu:

- Uma comunicação mais fluida e em tempo real;
- A eliminação de pedidos HTTP recorrentes ao servidor;
- Ao professor a capacidade de monitorizar a atividade dos alunos instantaneamente;

Esta implementação apresentou desafios adicionais, já que era a minha primeira experiência com WebSockets e com *backend* em Python. Contudo, após uma análise aprofundada e múltiplos testes, consegui concluir com sucesso esta funcionalidade.

5.1 Introdução ao Projeto

Na primeira semana do estágio, reuni-me com o professor Miguel Barão, que foi o meu coordenador ao longo deste período. Durante o encontro, o professor apresentou-me a plataforma *Perguntations*, destacando alguns problemas e possíveis melhorias que gostaria de implementar. Solicitou, ainda, a minha opinião, uma vez que já tinha utilizado a plataforma na Unidade Curricular de Arquitetura de Computadores I. Concordei com as questões levantadas e sugeri algumas funcionalidades adicionais.

Posteriormente, o professor pediu-me que, durante o resto da semana, consultasse os meus colegas para recolher mais opiniões e assim obter o maior número possível de contribuições. Assim, ao longo da semana, reuni-me com alguns colegas, registando as suas perspetivas para posteriormente as transmitir ao professor.

Na semana seguinte, após nova reunião com o professor, definimos as funcionalidades a desenvolver e as melhorias a implementar no projeto, nomeadamente:

- Introdução de novos tipos de questões (correspondência, preenchimento de lacunas e ordenação);

-
- Substituição do sistema de polling por WebSockets na página de administração;
 - Possibilidade de verificar a correção de questões de código durante o teste.

Durante esta reunião, o professor disponibilizou-me o código do projeto, alojado no GitLab. Para executá-lo, foi necessário instalar algumas ferramentas, como Python 3.11, pip (Gestor de pacotes do Python) e npm (Node Package Manager).

Após o encontro, dediquei o resto da semana ao estudo e análise do código, preparando-me assim para iniciar as implementações no projeto.

5.2 Desenvolvimento do projeto

5.2.1 Implementação dos tipos de questão

Depois de estudar o código, na semana seguinte tive uma reunião com o professor, durante a qual me foi dada a opção de escolher entre começar pelo *frontend* ou pelo *backend*. Decidi avançar com o *frontend*, uma vez que prefiro trabalhar nesta área e considero-a mais simples.

Como optei pelo *frontend*, isso significou que iria começar pela implementação dos novos tipos de perguntas. Já se tinha decidido que seriam adicionados exercícios do tipo **correspondência**, **ordenação** e **preenchimento de espaços**, uma vez que o *Perguntations* apenas incluía questões de escolha múltipla (*radio button* e *checkbox*) e questões de desenvolvimento ou código (*textarea*).

Na implementação dos tipos de questão, observei que todas seguiam um padrão comum, conforme verifiquei nos exemplos já desenvolvidos pelo professor no projeto. Cada tipo de questão requer:

- Um template HTML para renderizar a pergunta;
- Um template HTML para renderizar uma pergunta já respondida (modo de revisão);
- Uma classe em Python responsável por gerar e corrigir a questão;
- Um mecanismo de conversão dos dados provenientes do HTML para Python, permitindo a sua correção.
- Definição do tipo de pergunta em formato YAML.

Todos os templates HTML que renderizam as perguntas são templates Jinja2 [19] que herdam de um ficheiro base em comum, do projeto, - "*question.html*". Onde permitem a reutilização de elementos estruturais e estilísticos, como o rodapé, o botão para desativar a avaliação da questão e a respetiva pontuação.

Quanto ao template de revisão, também desenvolvido em Jinja2, este deriva de "*review-question.html*", também um ficheiro já existente no projeto, herdando componentes comuns como o rodapé e a cotação obtida na resposta.

As classes responsáveis pela criação e correção das questões estão implementadas num ficheiro, preexistente e concebido pelo professor. Este ficheiro oferece uma infraestrutura completa para gerar, apresentar, responder e corrigir automaticamente diferentes tipos de perguntas de avaliação, seguindo um formato estruturado em Python. Destacam-se dois métodos essenciais:

- **gen**: Inicializa os valores predefinidos da questão (definidos em YAML), e gera a pergunta.
- **correct**: Avalia a resposta do utilizador e calcula a nota correspondente.

Para garantir compatibilidade entre as respostas provenientes do HTML e o método **correct** em Python, foi necessário implementar no código do servidor um conversor específico para processar esses dados. Este conversor foi diferente para cada tipo de questão.

Todas as questões tiveram de ser configuradas num ficheiro YAML já existente, criado anteriormente pelo professor. Este ficheiro serve como modelo base para um sistema de criação de testes, configuração de parâmetros e avaliação automática. Para garantir o correto funcionamento, desenvolvi um exemplo prático para cada questão, o que me permitiu testar localmente no meu computador antes da implementação final.

Adicionalmente, foi necessário colocar num outro ficheiro YAML cada tipologia de questão elaborada, de modo a estabelecer uma cotação específica. Este ficheiro permite a atribuição automática de pontuação a

cada pergunta, permitindo ainda a possibilidade de priorizar questões (como, por exemplo, atribuir-lhes um peso superior ou inferior).

Implementação do tipo de questão de correspondência

Assim, decidi começar pela implementação do tipo de questão correspondência.

Para esta primeira implementação, discuti previamente com o professor, durante a reunião, o funcionamento deste tipo de questão e a metodologia de correção. Avaliámos diversas possibilidades, mas chegamos às seguintes conclusões:

- Cada opção da coluna da esquerda só pode corresponder a uma única opção da coluna da direita;
- As opções da coluna da direita, por outro lado, podem estar associadas a várias opções da coluna da esquerda;
- A cotação total da questão será distribuída igualmente pelo número de ligações existentes.

Com esta estrutura definida, avancei imediatamente para a definição do tipo de questão no ficheiro YAML onde foram criados dois grupos de itens que o utilizador deve associar.

Na template, HTML, de renderização da pergunta foram implementadas duas listas: uma que representa os itens a associar, e outra que representa os itens a serem associados. Os elementos da primeira lista são renderizados individualmente no lado esquerdo, com formatação Markdown [20]. À direita de cada elemento, é gerada uma caixa de seleção (dropdown) que, quando acionada, apresenta os itens da outra lista. Posteriormente, foi desenvolvido um script em JavaScript com a função de armazenar os dados no localStorage.

Relativamente ao template de revisão, este apresenta os elementos da lista da esquerda e verifica se o utilizador respondeu a cada uma das opções - as respostas do utilizador estão armazenadas numa lista própria. Caso o utilizador tenha respondido, a sua resposta é comparada com a lista de respostas corretas, sendo depois exibido um *feedback* de 'certo' ou 'errado', utilizando os ícones do Bootstrap. Quando o utilizador não responde a uma determinada opção, é apresentada uma mensagem a indicar que o item não foi respondido.

Quanto à classe em Python, no método *gen*, após extrair os valores do ficheiro YAML, são criadas duas sublistas (direita e esquerda) com base nos valores extraídos. Após a geração da pergunta, o método *correct* é invocado para avaliar a resposta e calcular a pontuação. Este processo consiste em analisar cada par de elementos. O sistema conta o número de pares corretos, sendo a pontuação calculada através da divisão entre o número de pares corretos e o número total de pares.

Para este tipo de questão, foi necessário implementar no código do servidor um conversor que transforma a lista proveniente do HTML, numa lista de inteiros, para poder ser avaliada no *backend* (Python).

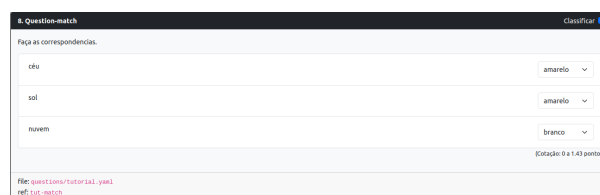


Figura 1: Resultado final da template de renderização para a questão



Figura 2: Resultado final da template de revisão para a questão

Implementação do tipo de questão de ordenação

O segundo tipo de questão a implementar foi o de ordenação. Tratando-se de um tipo de questão bastante simples, não foi necessário discutir pormenores específicos com o professor.

Na implementação do YAML para esta questão, foi definido um conjunto de opções, e o campo que representa a ordem correta dos elementos.

Para o template de renderização, foi criada uma lista ordenável visualmente, baseada nas opções definidas no YAML da questão, onde cada item pode ser arrastado. Sempre que o utilizador termina de mover um item, é acionado um callback (OnEnd) que serve para atualizar a nova ordem com os índices alterados. No template de revisão, é apresentada a lista final (resposta do utilizador) e, ao lado de cada opção, é mostrado um *feedback* de correto ou incorreto, utilizando os ícones do Bootstrap.

Na classe Python, o método *gen* inclui algumas exceções para verificar se as listas das opções e de correção são listas válidas, antes da criação da pergunta. Por sua vez, o método *correct* também valida se a lista das respostas do usuário é uma lista válida e corrige a questão com base no número de posições que foram acertadas pelo utilizador.

Para este tipo de questão, foi desenvolvido no servidor um conversor que processa os dados da requisição, transformando-os numa lista ordenada de valores inteiros.

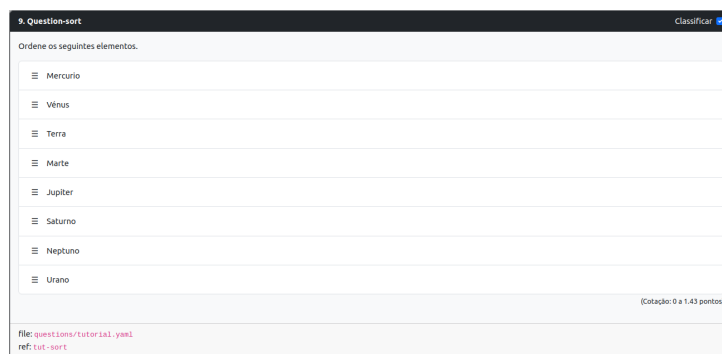


Figura 3: Resultado final da template de renderização para a questão

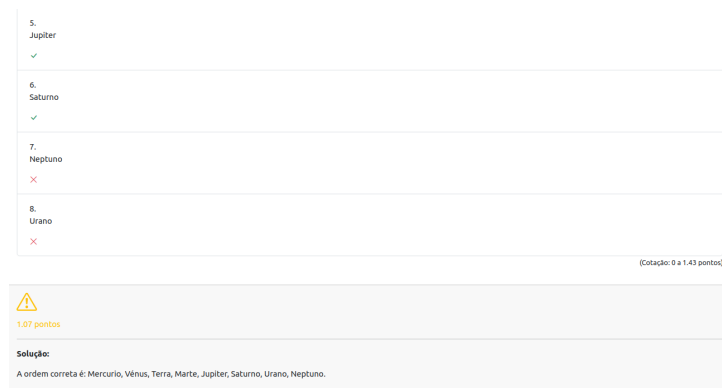


Figura 4: Resultado final da template de revisão para a questão (a imagem está cortada)

Implementação dos tipos de questões de preenchimento de espaços

O terceiro tipo de questão a implementar foi o de preenchimento de espaços. Antes de iniciar o desenvolvimento, reuni com o professor para definir a abordagem a adotar. Durante a discussão, levantámos várias hipóteses:

- Resposta direta numa caixa de texto.
- Seleção da opção correta num conjunto de alternativas (mecanismo de arrastar e largar).
- Utilização de um menu suspenso (dropdown) com várias opções.

- Combinação das abordagens anteriores numa única pergunta.

Após análise conjunta, optámos por implementar duas variantes distintas:

- Preenchimento por escrita (caixa de texto);
- Preenchimento por arrastar e largar (numa área com múltiplas opções).

Ambas as variantes seguiram um processo de desenvolvimento semelhante, diferenciando-se essencialmente nas templates HTML utilizadas.

Na implementação do YAML para estas questões, foram definidas frases (statements) que contêm um campo marcado com '@@', indicando que se trata de um espaço em branco a preencher, seguido da solução correta. No caso de questões em que o preenchimento é feito por arrastamento da resposta correta, foi necessário incluir um campo com opções (options).

Relativamente aos templates de renderização, para este tipo de questões, foi necessário, em ambos os casos, dividir cada linha dos enunciados utilizando o símbolo ":@" como delimitador. A função deste marcador é indicar a posição onde deverá ser inserido um espaço em branco a preencher. Nestas posições, é gerada uma caixa de texto que pode ser preenchida de formas diferentes dependendo do template:

- Preenchimento por texto – O utilizador introduz manualmente a resposta.
- Preenchimento por arrastamento – O utilizador seleciona uma das opções disponíveis e arrasta-a para a caixa de texto designada (*dropzone*).

Para este último caso, foi criada uma área específica contendo as diferentes opções definidas (*draggables*), as quais podem ser arrastadas e largadas nas respetivas *dropzones*.

Os templates de revisão são idênticos para ambos os tipos. Em ambos, é apresentado o enunciado com uma caixa de texto no lugar do marcador '@@', onde se encontra o input introduzido pelo utilizador e onde não é possível alterar a resposta. Ao lado da caixa, surge um *feedback* que indica se a resposta está correta ou incorreta.

As classes em Python também são semelhantes. Em ambos os casos, ao gerar a pergunta, verifica-se se o número de lacunas corresponde ao número de respostas e, no preenchimento por arrastamento, confirma-se se 'options' existe e é uma lista. Em qualquer dos cenários, o *correct* é igual, uma vez que a cotação corresponde à média das respostas certas face ao total de respostas.

No servidor, as respostas são decodificadas para texto UTF-8 [21] para a correção puder ser feita.

Figura 5: Resultado final da template de renderização para a questão de preenchimento por texto

Figura 6: Resultado final da template de renderização para a questão de preenchimento por arrastamento

Figura 7: Resultado final da template de revisão que é igual para ambos os tipos

Implementação dos tipos de questões de correspondência com linhas

Por último, o tipo de questão a ser implementado foi uma variação do tipo de correspondência com menus. Esta variação segue a mesma lógica, mas as associações são feitas através de ligações. A implementação deste tipo só foi realizada nessa fase porque foi quando eu e o professor nos lembramos de a incluir.

A estrutura desta variação é muito semelhante à do tipo de correspondência original. As únicas diferenças estão nos templates de renderização e na revisão do HTML – todo o resto é idêntico.

Na template de renderização, é criada uma interface visual com duas colunas (esquerda e direita), onde o utilizador pode clicar num item de cada lado para estabelecer uma ligação entre eles. As ligações são representadas através de linhas tracejadas em SVG [22] entre os elementos correspondentes. O resultado destas ligações é armazenado num campo (input hidden) no formato JSON [23].

Esta interface permite que um elemento da coluna da esquerda se ligue apenas a um elemento da coluna da direita. Contudo, existe uma secção no código, que encontra-se comentada, e que possibilita que um elemento da esquerda se ligue a múltiplos elementos da direita. O restante da lógica desta template é idêntico ao da outra template de correspondência.

Na template de revisão, são apresentadas as ligações SVG realizadas pelo utilizador, as quais podem aparecer a verde (correcto) ou a vermelho (errado).

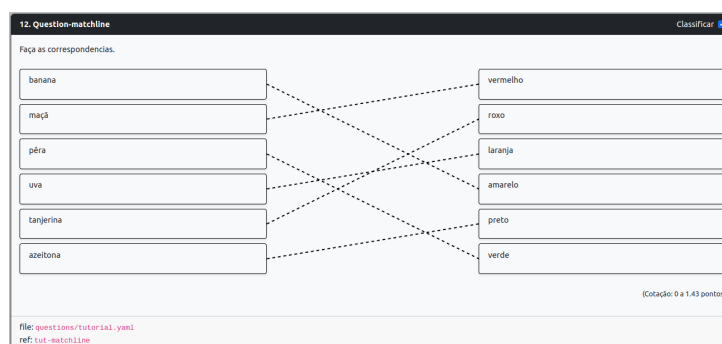


Figura 8: Resultado final da template de renderização para a questão

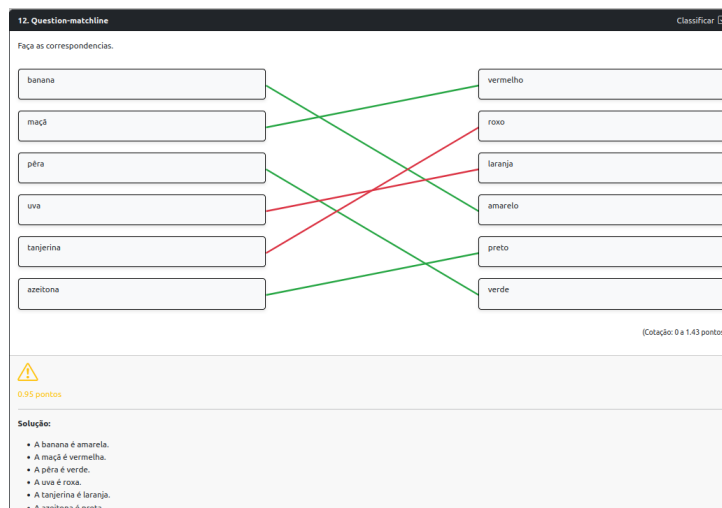


Figura 9: Resultado final da template de revisão para a questão

5.2.2 Implementação do verificador de resposta

Após concluir a implementação dos tipos de questão no *frontend*, que demorou pouco mais de um mês, avancei para a segunda fase do trabalho: o desenvolvimento do *backend*. Nesta etapa, concentrei-me em duas tarefas principais, começando pela criação de um avaliador de respostas (para código ou texto), que funciona carregando num botão durante a realização do teste.

Para implementar esta funcionalidade, foi necessário modificar a template HTML já existente, criada pelo professor, que exibe uma questão com uma área de texto para respostas de escrita ou código. A template original incluía o botão, mas sem qualquer ação associada. Desta forma, adicionei um script que define a função executada quando o botão é pressionado. Esta função envia a resposta do utilizador, (o texto inserido por ele na área de texto), para um endpoint através de um pedido AJAX POST.

Foi desenvolvido um ficheiro em Python com Tornado que funciona como uma API (endpoint POST). Este recebe uma resposta em formato JSON, analisa o seu conteúdo (verificação de sintaxe e/ou testes unitários) e devolve um *feedback* estruturado, incluindo uma cor e uma mensagem, também em formato JSON. Posteriormente, o JavaScript processa esta resposta e apresenta-a ao utilizador.

Adicionalmente, o botão foi programado com um limite de uso, que pode ser ajustado diretamente no código da template, garantindo um controlo sobre o número de submissões permitidas.

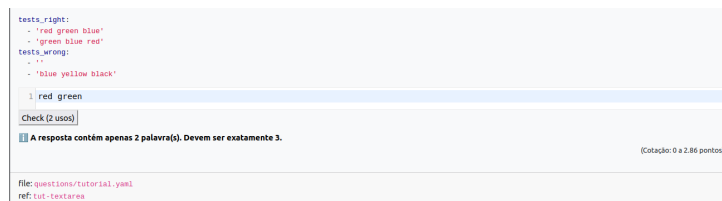


Figura 10: Resultado final após usar 1 uma vez (neste caso o limite de usos são 3 no total)

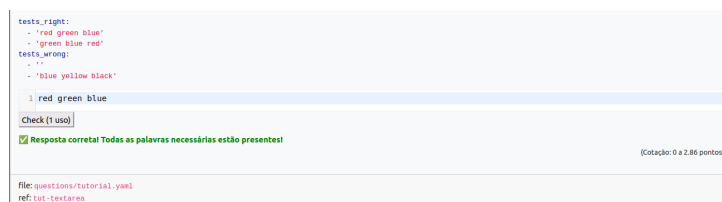


Figura 11: Resultado final após usar uma segunda vez

5.2.3 Substituição do polling por WebSockets na página de administração

A segunda tarefa no *backend* focou-se na migração do sistema de polling para WebSockets na página de administração da plataforma, que monitoriza em tempo real as ações dos utilizadores durante os testes, como o tamanho do ecrã, saída da página, início e conclusão do teste. Esta tarefa foi a última a ser desenvolvida, e demorou cerca de 3 semanas, tal como a tarefa anterior.

O método anterior utilizava pedidos AJAX repetidos a cada 3 segundos, o que gerava tráfego desnecessário, sobrecarregava o servidor com múltiplos utilizadores e afetava o desempenho do cliente devido à renderização constante. Além disso, introduzia um atraso nas atualizações.

Com a implementação de WebSockets, eliminaram-se os pedidos HTTP recorrentes, reduzindo a carga no servidor e no navegador. Agora, as atualizações são instantâneas e o código está mais organizado, com uma distinção clara entre a inicialização da conexão e o tratamento de eventos. Esta mudança melhorou significativamente a eficiência e a responsividade da página de administração.

Para implementar os WebSockets, criei uma função que estabelece a ligação e gere os seguintes eventos:

- **onopen**: envia os comandos iniciais;
- **onmessage**: recebe e processa as respostas;
- **onclose/onerror**: regista logs e emite alertas em caso de erro.

Quanto à DataTable, que anteriormente era preenchida via AJAX e formatava os dados recebidos do servidor em cada chamada, agora é inicializada vazia. Quando os dados chegam através do WebSocket, a função `updateStudentsTable(data.data)` atualiza a tabela manualmente (no `onmessage`).

6 Avaliação crítica

Ao longo deste projeto, considero ter aprendido bastante, não só por ter trabalhado com ferramentas novas, mas também por ter aprofundado o conhecimento em tecnologias que já dominava, como o Python. Neste caso, explorei diferentes abordagens e bibliotecas, o que me permitiu expandir as minhas competências. No que diz respeito ao *frontend*, apesar de já ter experiência com HTML, CSS e JavaScript, foi a primeira vez que utilizei templates Jinja2 e as integrei com YAML, uma tecnologia com a qual também não tinha trabalhado anteriormente.

Gostei imenso de desenvolver esta plataforma e acredito que o seu impacto vai além do meu percurso académico. Além de ter sido uma experiência enriquecedora a nível de aprendizagem, penso que será um recurso valioso para a universidade, tanto para professores como para futuros alunos, na realização de provas de avaliação. Espero que, agora mais completa, possa ser adotada por mais docentes em diversas Unidades Curriculares. Confesso que lamento não ter conseguido implementar mais funcionalidades, devido às limitações de tempo do estágio, mas acredito que o essencial foi concretizado.

Ao longo deste período, percebi a importância do trabalho em equipa. Apesar de não ter realizado o estágio num ambiente empresarial, onde essa dinâmica é mais intensa, o acompanhamento próximo do professor foi fundamental. Durante as nossas reuniões, discutimos detalhadamente a implementação de várias funcionalidades, e esse diálogo foi crucial para o sucesso do projeto. Sempre que surgiam dúvidas, o professor estava disponível para ajudar, e juntos encontrávamos as melhores soluções.

Quanto às aprendizagens, agradeço ao professor pelos conhecimentos partilhados ao longo das nossas reuniões. Muitas vezes, aprendi conceitos que, embora não estivessem diretamente relacionados com o projeto, foram extremamente úteis para a minha formação. Por exemplo, o professor mostrou-me alguns projetos desenvolvidos por ele em Haskell e Rust, o que me permitiu ter um primeiro contacto com estas linguagens — uma experiência muito interessante e enriquecedora. Além disso, numa das primeiras reuniões, ensinou-me alguns comandos de Linux, uma área em que tinha pouco domínio, mas que agora utilizo com frequência.

Por fim, posso dizer que fiquei muito satisfeito com o resultado deste projeto. Acredito que realizei um bom trabalho e adquiri conhecimentos que serão valiosos no futuro. Além disso, considero que a colaboração com o professor foi muito boa, marcada por uma dinâmica comunicativa e produtiva.

Referências

- [1] Css. <https://www.w3schools.com/css/>.
- [2] Websockets. <https://websockets.readthedocs.io/en/stable/>.
- [3] Argon2. <https://argon2-cffi.readthedocs.io/en/stable/argon2.html>.
- [4] Mistune. <https://mistune.lepture.com/en/latest/>.
- [5] Pyyaml. <https://pypi.org/project/PyYAML/>.
- [6] Pygments. <https://pygments.org/>.
- [7] Sqlalchemy. <https://www.sqlalchemy.org/>.
- [8] Tornado web framework. <https://www.tornadoweb.org/en/stable/>.
- [9] Bootstrap. <https://getbootstrap.com/docs/3.4/javascript/>.
- [10] Codemirror. <https://codemirror.net/>.
- [11] Datatables. <https://datatables.net/>.
- [12] jquery. <https://jquery.com/>.
- [13] Mathjax. <https://www.mathjax.org/>.
- [14] Sortablejs. <https://sortablejs.github.io/Sortable/>.
- [15] Dom - document object model. https://www.w3schools.com/whatis/whatis_htmlDOM.asp.
- [16] Ajax. https://www.w3schools.com/Js/js_ajax_intro.asp.
- [17] Yaml. <https://www.redhat.com/en/topics/automation/what-is-yaml>.
- [18] Desenvolvimento Ágil de software. https://en.wikipedia.org/wiki/Agile_software_development.
- [19] Jinja2. <https://pypi.org/project/Jinja2/>.
- [20] Markdown. <https://www.markdownguide.org/>.
- [21] Utf-8. <https://pt.wikipedia.org/wiki/UTF-8>.
- [22] Svg - scalable vector graphics. <https://en.wikipedia.org/wiki/SVG>.
- [23] Json - javascript object notation. <https://www.json.org/json-en.html>.